VisionGuard Assignment: AI vs Real Image Classification and Deployment

1. Overview and Rules

You will build **VisionGuard**, a web service that tells whether an uploaded image is AI-generated or real. You will train a TensorFlow/Keras model, track every experiment with MLflow on DagsHub, serve the model through a Flask web app, log every request in a database, containerize everything with Docker Compose, and deploy it to the cloud.

Deliverables: (1) a PDF report with screenshots, (2) a GitHub repository, (3) a public or shared DagsHub project, (4) a live URL. Marks total 100 (Section 10).

Rules

- You may use any cloud (Oracle, Vercel, Google, AWS, Azure) for Task 7.
- Create Dagshub Repository for code, data and model versioning.
- Project Name is Your **[Roll Number_A2_Ops]** (Everywhere) Cloud/DAGSHUB etc.

2. Dataset

Dataset: AI Generated Images vs Real Images (https://www.kaggle.com/datasets/cashbowman/ai-generated-images-vs-real-images) on Kaggle. It is a two-class collection: images scraped from websites (**real**) and images produced by generative AI (**AI-generated**), covering subjects such as people, animals, portraits and landscapes. The label of an image is the folder it sits in.

3. Task 1: Data Understanding and Versioning (7 marks)

Goal: prove you understand the data before training anything.

ID	Sub-task and instructions	Evidence for report	Marks
1.1	**Environment.** Create an isolated Python environment and install TensorFlow plus the libraries you need. Record exact versions and check whether TensorFlow can see a GPU.	Terminal screenshot of versions	1
1.2	**Dataset audit.** Write your own script that walks the dataset and builds a table with, for every image: class, file size, width, height, colour mode, format and a content hash (SHA-256). Detect unreadable files and exact duplicates.	Table of counts per class; min, median and max of width, height and file size; number of corrupt files and duplicates	3
1.3	**Visual EDA.** Show a grid of sample images per class and histograms of width, height and file size per class. Write a short interpretation of each figure.	Figures with captions and 2 to 3 sentences each	1.5

ID	Sub-task and instructions	Evidence for report	Marks
1.4	**DVC (data version control).** *Toy part:* in a new empty folder with a small CSV, initialise Git and DVC, track the CSV with DVC, change it, commit both versions(dagshub), then return to the older version. Configure a remote storage, push, delete the local data, and pull it back. *Apply:* track your dataset folder the same way, with DagsHub's storage or a local folder as remote. Explain what the small pointer file holds and why large data does not belong in Git.	Screenshots of initial commits	1.5

4. Task 2: Model Development with TensorFlow/Keras (14 marks)

Goal: train two models, a CNN built from scratch and a pretrained lightweight model fine-tuned on your data, and compare them.

ID	Sub-task and instructions	Evidence for report	Marks
2.1	**Input pipeline.** Build a `tf.data` pipeline that reads your split lists, decodes and resizes images, batches and prefetches them. Apply random augmentation to the training data only. Explain every stage, the image size you chose, and what happens to aspect ratio.	Description of the pipeline, a figure of one augmented batch	2
2.2	**Baseline CNN.** Design a small CNN yourself with Keras layers (convolution, pooling, normalization, dropout, one sigmoid output). Choose loss, optimizer and metrics and justify them. Explain why each layer is there.	Model summary screenshot, table of output shape and parameter count per layer	3
2.3	**Transfer learning.** Load a pretrained model from `keras.applications` (MobileNetV2, EfficientNetB0 or ResNet50) without its classifier. Freeze it, add your own head and train. Then unfreeze only the last block and train again with a much smaller learning rate. Use the preprocessing that your chosen pretrained model expects and explain why.	Two sets of training curves, list of trainable layers in each stage	3
2.4	**Training discipline.** Fix random seeds. Use early stopping, model checkpointing on validation loss and learning-rate reduction callbacks. State the best epoch.	Loss and accuracy curves with best epoch marked	1

ID	Sub-task and instructions	Evidence for report	Marks
2.5	**Evaluation.** Evaluate on the test set **once**: accuracy, precision, recall, F1, ROC-AUC, confusion matrix and ROC curve. Choose the decision threshold using validation data and justify it. Decide which error is worse for this product: calling a real photo fake, or a fake photo real.	Metric table for both models, figures, written justification	2
2.6	**Error analysis.** Show 10 misclassified test images and a hypothesis for each (for example blur, painting style, watermark, size).	Image grid and short analysis	1
2.7	**Tool lab: Keras Tuner.** *Toy part:* tune a tiny network on a built-in dataset such as Fashion-MNIST, searching learning rate and number of units with random search or Hyperband, and read the best settings. *Apply:* tune learning rate, dropout and number of unfrozen layers for your model with at least 8 trials. Say which setting mattered most and why choosing the best validation score can overfit the validation set.	Toy results table, tuner results for VisionGuard, discussion	2

Bonus (up to +1, capped): Grad-CAM heat maps for 4 correct and 4 wrong predictions, with a comment on what the model looks at.

5. Task 3: Experiment Tracking with MLflow and DagsHub (12 marks)

Goal: every run must be reproducible and comparable. Anyone opening your DagsHub project should see what you tried, what it scored and which model you chose.

ID	Sub-task and instructions	Evidence for report	Marks
3.1	**Tool lab: MLflow locally.** Train a tiny Keras model on a built-in dataset. Log parameters, per-epoch metrics, one plot and the model, then open the local MLflow interface and compare two runs with different learning rates. Define experiment, run, parameter, metric, artifact and tag in your own words.	Screenshots of both runs and the comparison view, your definitions	2
3.2	**DagsHub setup(IF not created in Task 1).** Create a DagsHub project connected to your GitHub repository. Find its MLflow tracking address, create an access token and give both to your code through environment variables, never by writing them in code. Repeat the toy run against DagsHub.	Screenshot of the DagsHub project, the toy run on DagsHub, proof that `.env` is git-ignored	2

ID	Sub-task and instructions	Evidence for report	Marks
3.3	**Instrument VisionGuard training.** Log hyperparameters, per-epoch training and validation metrics, final test metrics and tags (git commit, DVC data version, seed). Do it once manually (for example with a Keras callback) and once with MLflow's TensorFlow autologging, and explain what each approach captures or misses.	Run page showing parameters, metric curves and tags for both approaches	3
3.4	**Artifacts.** Attach confusion matrix, ROC curve, the split lists, the package list and the trained Keras model to the run.	Artifacts tab screenshot	1.5
3.5	**Experiment comparison.** Produce at least 6 runs where each differs from another in **one** meaningful change (learning rate, augmentation, frozen layers, image size, backbone). Compare them in one table or chart, choose the best, and justify the choice with the metrics you consider most important.	Comparison screenshot, written justification	2
3.6	**Model registry.** Register your best model, create at least two versions, give one the alias `champion`, then write a separate small script that loads the model by alias and classifies one image.	Registry screenshot, terminal output of the load and prediction	1.5

The serving app must **not** call DagsHub for every request. Plan how the champion model file reaches your server (Task 6).

6. Task 4: Flask Web Application (12 marks)

Goal: a user uploads an image in a browser and sees a verdict, a confidence value and an honest caveat.

ID	Sub-task and instructions	Evidence for report	Marks
4.1	**Inference module.** Write a separate module that loads your champion Keras model **once** at start-up, preprocesses an uploaded image exactly as in training, and returns a label and a confidence. Explain why loading per request is unacceptable and why training and serving preprocessing must match.	Short explanation, console output for two known images	2
4.2	**Routes and validation.** Create pages for upload, result, history and a health check. Reject wrong file types, files over a size limit and files that look like images but cannot be decoded. Sanitize file names. Do not keep the uploaded image on disk. Show	Screenshots of the three failure cases	3

ID	Sub-task and instructions	Evidence for report	Marks
	friendly error pages.		
4.3	**HTML and CSS.** A simple html css template where user may be able to upload image for model inference, model will return response and will be displayed. A history in menu to show model history upon click (from database)	Frontend	2.5
4.4	**Configuration and production server.** Read all settings (model path, threshold, database address) from environment variables. Run the app in flask server and with debug mode off. Explain why.	Health-check response screenshot, explanation	1

7. Task 5: Database Trail (8 marks)

Goal: every request leaves a permanent, queryable record that answers: what was decided, when, by which model, and how confidently.

You have to create a database (SQLITE/Redis/Postgres) and must attach in Flask views to keep history of the searches. Whenever, any search is made and model returns the results, the entry should be made in database. There should be a history menu in HTML, CSS where users may be able to view history of the searches, also what was decided, when, by which model, and how confidently.

8. Task 6: Containerization with Docker and Docker Compose (14 marks)

Goal: anyone with Docker can run your whole system with one command and get identical behaviour.

ID	Sub-task and instructions	Evidence for report	Marks
6.1	**Docker warm-up.** Before touching your project, practise with a ready-made image: run a container, map a port, mount a volume, read its logs, open a shell inside it, list and remove containers and images. Define image, container, layer, volume and port mapping in your own words.	Screenshot of each action and your definitions	2
6.2	**Dockerfile for the web app.** Write it so that dependencies are installed before the source code is copied (explain the layer cache), use a slim base, run as a non-root user, add a health check and an ignore file, and keep secrets out of the image. TensorFlow images are large: choose your TensorFlow package deliberately (CPU-only is enough) and justify it.	Dockerfile link, explanation of the order of instructions, output showing the user inside the container	3

ID	Sub-task and instructions	Evidence for report	Marks
6.3	**Compose stack.** Define two services, the web app and the database. Services must reach each other by service name on Compose's internal network. The database uses a named volume, waits for readiness through a health check before the web app starts, and **publishes no port** to the outside. Read all passwords from an untracked `.env` file and provide `.env.example`. Decide how the model file reaches the container (mounted or built in) and justify it.	Compose file link, service status showing healthy, proof that the database port is closed from outside	4
6.4	**Image analysis.** Report the image size and which layers are largest. Apply one optimization (for example a smaller base, no pip cache, ignore file, or multi-stage build) and report before and after sizes.	Two size numbers and a conclusion	1.5
6.5	**Operations and troubleshooting.** Demonstrate start, stop, restart, logs and a shell inside each service. Then break something on purpose (wrong database password or missing model file), diagnose it only from logs, and fix it.	Log excerpt of the failure, diagnosis, fix	1.5
6.6	**Tool lab: GitHub Actions (CI).** *Toy part:* in a throwaway repository create a workflow that runs on every push, uses a matrix of two Python versions and runs a small test file. Make a test fail to see a red run, then fix it for a green run. *Apply:* create a workflow for VisionGuard that installs dependencies, runs linting and your tests, and builds the Docker image. Add the status badge to the README. State where workflow secrets are stored and why they must not appear in the workflow file.	Red and green toy runs, green project run, badge in README	2

9. Task 7: API Deployment (15 marks)

Goal: the model api is reachable from the public internet, from a network other than yours. Choose **one** pathway; the same marks apply to all.

Your task is to put model on a cloud/vercel and make api calls from flask. Flask(Itself) code will be running on localhost.

Pathway	What you deploy	Things to research first
A. Oracle Cloud (OCI Always Free)	The whole Compose stack on one Ubuntu VM	Current free limits shown in *your* console, Arm versus x86 images, free-instance availability, idle-instance reclamation, the two firewalls (cloud network rules and the VM's own)
B. Vercel with external inference	Flask UI on Vercel; model server and database elsewhere	Vercel function size limit and lack of persistent disk (TensorFlow will not fit, SQLite will not persist), request size limit, timeouts, protecting the model API with a secret key, a managed PostgreSQL service
C. Any other cloud (Google, AWS, Azure)	A VM running Compose, or a managed container service plus managed PostgreSQL	Current free-tier terms, mapping each Compose element to the provider's equivalent

- Test your api in the postman and document the logged result in report.
- Add step by step instructions on model deployment with cloud/vercel along screenshots in the report.
- ADD API URL in the report too.

10. Task 8: Integrating Flask and API (15 marks)

Integrate flask view with API(Hosted on Cloud/Vercel) to demonstrate its working. Send multiple inputs to model and record its response with screenshot. Also log history menu to show database logs.

11. Report

11.1 Report format (PDF)

- **Front matter:** name, roll number, links (GitHub, DagsHub, live URL), commit hash of the graded version, and a disclosure of any AI assistant or tutorial you used and for what.
- **One chapter per task.** Under each sub-task ID write what you did, *why* you did it, how it works in your own words, and what went wrong. Explain every command, setting and design choice; do not paste code or screenshots without commentary.
- **Screenshots:** every sub-task needs the evidence listed in its table. Each screenshot must be captioned, referenced in the text, and show enough context (address bar or terminal prompt with your username, date or time, project name) to prove it is your work.

11.2 Submission checklist

- PDF report named `<RollNumber>_VisionGuard.pdf`, within the page limit your instructor sets
- DagsHub project link
- API URL tested from another network